Linguaggi

Università di Verona
Imbriani Paolo -VR500437
Prof.ssa Isabella Mastroeni

March 4, 2026

Contents

1 Introduzione **3**

2 Linguaggi di programmazione **3**

2.1 Linguaggi formali 4

2.2 Aspetti di progettazione 4

2.3 Tipi di linguaggi di programmazione 5

2.3.1 Paradigmi di programmazione 5

3 Implementazione di un linguaggio di programmazione **6**

3.1 Macchina astratta per il linguaggio di programmazione 6

3.2 Interprete 7

3.2.1 Struttura dell'interprete 7

1 Introduzione

Un linguaggio interpretato è un linguaggio è un linguaggio di programmazione in cui le istruzioni vengono eseguite direttamente, senza la necessità di una fase di compilazione. Un interprete è rappresentato da un modello di esecuzione cioè un CFG che rappresenta il flusso di controllo del programma. L'obiettivo è di studiare i linguaggi di programmazione e capire cosa li accomuna (insieme alle loro macro-differenze) e quale è il loro ruolo all'interno dello sviluppo. L'idea ora è capire quali cambiamenti sono stati necessari introdurre per arrivare a linguaggi di programmazione più evoluti, e capire quali sono i problemi che si sono incontrati nel corso del tempo e come sono stati risolti.

2 Linguaggi di programmazione

Da una parte abbiamo:

- **Dati**: che può essere qualsiasi cosa, numeri, stringhe qualcosa da manipolare
- **Algoritmi**: ovvero il calcolo da eseguire sui dati

Definizione 2.1: Linguaggio di programmazione

Un **linguaggio di programmazione** è uno strumento formale per descrivere algoritmi applicati ai dati.

Definizione 2.2: Programma

Un **programma** è una rappresentazione sintattica (che rispetta regole sintattiche del linguaggio di programmazione) finita di comportamenti potenzialmente infiniti. Questi comportamenti vengono definiti attraverso delle funzioni:

$$f : Dom \to Cod \cup \{\uparrow\}$$

(la funzione calcolata dall'algoritmo rappresentata/implementata attraverso il programma).

Descriviamo prima però cosa è un algoritmo:

Definizione 2.3: Algoritmo

Sequenza di passi primitivi finita descritta attraverso un programma in un linguaggio di programmazione.

2.1 Linguaggi formali

Esistono diversi tipi di linguaggi formali:

- **Linguaggio matematico**: notazione rigorosa per rappresentare funzioni:

  $$f(x) = x + 2$$

  Da questo linguaggio è nato un paradigma quello **funzionale** (λ-calcolo, ML, Haskell, Lisp...)

- **Linguaggio logico**: notazione rigorosa, regole e assiomi, calcolo come dimostrazione. Questo linguaggio però manca la descrizione di calcoli potenzialmente infiniti. Anche da qua nasce un paradigma, quello **logico** (Prolog).

- **Linguaggio di programmazione**: notazione rigorosa (grammatica), regole (semantica delle istruzioni), istruzioni che permettono di descrivere i passi di calcolo.

Dalle esigenze di diversi tipi di applicazioni, sono nati diversi tipi di linguaggi di programmazione:

- Applicazioni scientifiche come **FORTRAN**
- Applicazioni economiche come **COBOL**
- Applicazioni AI come **LISP**
- Applicazioni di sistema come **C**
- Applicazioni web come **Javascript, Java, ...**

2.2 Aspetti di progettazione

Uno degli aspetti più importanti nella progettazione di un linguaggio di programmazione sono:

- **Readability**:
  - Sintassi chiara
  - No ambiguità
  - Facilità di lettura che rende facile comprendere i programmi
  - Facilità di analisi del codice
- **Writeability**:
  - Facilità di scrittura, quanto è facile/agevole
  - Scrivere programmi in quel linguaggio, facilità di analisi del codice
- **Reliability**:

  - Conformità alle specifiche
  - Riduzione di inserimento di errori di programmazione

- **Cost**:
  - Costo di efficienza
  - Costo di apprendimento
  - Costi di utilizzo in modo corretto
  - ...

2.3 Tipi di linguaggi di programmazione

Ci sono diversi tipi di linguaggi di programmazione:

- **Linguaggi di basso livello**: vicini al linguaggio macchina come:
  - Linguaggio macchina
  - Linguaggio assembly
- **Linguaggi di alto livello**: più vicini al linguaggio umano come:
  - Linguaggi funzionali
  - Linguaggi logici
  - Linguaggi imperativi (l'unione con i linguaggi funzionale si chiama "funzionali impuri") dove vengono definite variabili come astrazione di una cella di memoria

In questi linguaggi le variabili:

- Nei linguaggi **imperativi** è un'astrazione di una cella di memoria, che può essere modificata
- Nei linguaggi **funzionali** e **matematica** è una variabile matematica e quindi **immutabile**

2.3.1 Paradigmi di programmazione

Nei linguaggi ci sono i seguenti paradigmi:

- **Object-oriented**: dove i programmi sono organizzati in classi che rappresentano oggetti, con attributi e metodi.
- **Procedurali**: dove i programmi sono organizzati in procedure o funzioni che eseguono compiti specifici.
- **Strutturata**: dove i programmi sono organizzati in blocchi di codice
- **Dichiarativi**: si basano sulle basi matematiche e logiche, dove i programmi sono organizzati in dichiarazioni che descrivono ciò che deve essere fatto, piuttosto che come farlo.

3 Implementazione di un linguaggio di programmazione

Per implementare un linguaggio di programmazione, è necessario:

- **Architettura** della macchina su cui è implementato
- **Metodologie di progettazione**: cambiare da un linguaggio ad oggetti a un linguaggio dichiarativo determina cambiamenti nella progettazione del linguaggio e quindi anche nella sua implementazione

3.1 Macchina astratta per il linguaggio di programmazione

Abbiamo bisogno di una macchina astratta per il linguaggio di programmazione, che è un modello di esecuzione che rappresenta il flusso di controllo del programma.

Definizione 3.1: Macchina astratta

Dato un LP L, una M_L (macchina astratta per L) è un insieme di **strutture dati** e **algoritmi** che permettono di memorizzare e eseguire programmi scritti in L. Intuitivamente, M_L è la macchina il cui linguaggio è esattamente L.

Definizione 3.2: Linguaggio macchina

Dato una macchina M il linguaggio L_m (linguaggio macchina di M) è il linguaggio che ha come stringhe quelle direttamente interpretabili da M.
Si ha una pila di macchine astratte M_{i-1}, M_i, M_{i+1} e i linguaggi macchina $L_{m_{i-1}}, L_{m_i}, L_{m_{i+1}}$. Il linguaggio L_{m_i} può essere usato ai livelli superiori.

Se immaginiamo le macchine astratte a più livelli, le macchine astratte di livello più basso donano i loro servizi ai livelli superiori che usano i linguaggi ad alto livello per implementare funzioni più complesse. Questo compongono diversi **livelli di astrazione**.

$$\text{Implementare } L \equiv \text{Realizzare } M_L$$

Per realizzare M_L si può:

- Realizzare M_L vuol dire tradurre L in un linguaggio macchina sottostante. (Compilazione)
- Oppure si può eseguire L sulla macchina sottostante. (Interpretazione)

Definizione 3.3: Compilazione

Dati due linguaggi L, L' una **compilazione** vuol dire fare una traduzione esplicita tra L e L' o viceversa.

Definizione 3.4: Interpretazione

Dati due linguaggi L, L' una **interpretazione** vuol dire fare una traduzione implicita tra L in L' ovvero simuliamo l'esecuzione di L attraverso L'.

3.2 Interprete

Per definire cosa è un interprete dobbiamo introdurre le seguenti notazioni:

- $Prog^L$: insieme dei programmi scritti in L
- D: insieme dei dati manipolati dai programmi in L
- P^L un singolo programma in L. (Quindi $P^L \in Prog^L$)

Definizione 3.5

Dato L un linguaggio di programmazione, un **interprete** per L (scritto in L_0) è un programma

$$I^{LoL} : (Prog^L \times D) \to D$$

L'inteprete è la macchina di Turing universale.

$$\underbrace{I^{LoL}(P^L, input) = output \text{ t.c } P^L(input) = output}_{I^{LoL}(P^L, input) = P^L(input)}$$

3.2.1 Struttura dell'interprete

Vediamo ora come un generico interprete è strutturato, e quali sono le sue componenti principali. Lo scriveremo in pseudocodice, e vedremo che è composto da tre fasi principali: Fetch, Decode, Execute.

```
go := true;
while go do
    fetch (opcode, opinfo) from PC;
    decode (opcode, opinfo);
    if opccode needs arguments
        then fetch arguments from PC;
    case opcode of
        opcode1: execute opcode1;
        opcode2: execute opcode2;
        ...
        opcodeN: execute opcodeN;
        halt: go := false;
```

```
if opcode has res then store(res)
pc = pc + sizeof(opcode) + sizeof(arguments);
```